

PROGRAMAÇÃO DE COMPUTADORES



Manipulação de Arquivos em Python

Prof. Dr. José Jairo de Santana e Silva

Leitura, escrita, CSV, dicionários e relatórios

Manipulação de Arquivos em Python: programas que lembram dados depois que fecham

AGENDA DA AULA

☰ Agenda para 4 horas

Hoje a aula é **prática**. A ideia é sair sabendo ler dados, processar com dicionários e gerar arquivos de saída.

Bloco	Foco
1	Por que arquivos? Revisão de listas e dicionários
2	Arquivos .txt : ler, escrever e acrescentar
3	Arquivos .csv : linhas, colunas e conversões
4	csv.DictReader : cada linha como dicionário
5	csv.DictWriter : gerar relatórios
6	Miniatividade orientada
7	Fechamento e revisão do fluxo completo

OBJETIVOS DE APRENDIZAGEM

🎯 Objetivos

Ao final da aula, o estudante deve ser capaz de:

- Abrir arquivos com `with open(...)`
- Ler conteúdo de arquivos `.txt`
- Escrever e acrescentar conteúdo em arquivos
- Ler arquivos `.csv` com o módulo `csv`
- Transformar linhas de CSV em **dicionários**
- Guardar registros em uma **lista de dicionários**
- Converter campos de texto para `int` e `float`
- Gerar arquivos `.csv` e `.txt` como saída
- Produzir um relatório simples a partir de dados

POR QUE ARQUIVOS?

Variáveis somem quando o programa acaba

Quando usamos apenas variáveis:

```
alunos = [  
    {"nome": "Ana", "nota": 8.0},  
    {"nome": "Bruno", "nota": 6.0}  
]
```

os dados ficam na memória **enquanto o programa está rodando**.

Se fechar o programa, os dados somem.

Arquivo é uma forma simples de **persistência**: guardar dados no disco para usar depois.

Onde isso aparece na prática

Ler e escrever arquivos é uma das tarefas **mais comuns** em programação:

- **Planilhas exportadas:** Excel/Google Sheets → "Salvar como CSV" gera o arquivo que vamos ler.
- **Datasets públicos:** IBGE, dados.gov.br, Kaggle, Banco Central. Quase tudo vem em **.csv**.
- **Logs de sistemas:** servidores escrevem **.log** (texto) com tudo que acontece; engenheiros leem para depurar.
- **ETL (Extract, Transform, Load):** ler arquivos brutos, transformar, gravar resultado limpo.
- **Configurações:** **settings.json**, **config.yaml**, **.env**. Programas leem esses arquivos ao iniciar.
- **Backups:** dumps de banco de dados em **.sql** ou **.csv** são guardados periodicamente.
- **Integrações:** sistemas trocam dados via CSV/JSON em pastas compartilhadas ou FTP.

■ Mesmo em sistemas com banco de dados, os dados normalmente entram e saem por arquivos.

O caminho da aula

Hoje vamos seguir este fluxo:

```
arquivo.csv
|
v
ler com Python
|
v
lista de dicionários
|
v
processar dados
|
v
resultado.csv + resumo.txt
```

Esse é o fluxo que vamos praticar ao longo da aula.

Ponte com a aula de dicionários

Na aula anterior, vimos dados assim:

```
aluno = {  
    "nome": "Ana",  
    "matricula": "2026001",  
    "nota": 8.5  
}
```

Hoje vamos ler isso de um arquivo:

```
nome,matricula,nota  
Ana,2026001,8.5  
Bruno,2026002,6.0
```

E transformar cada linha em um dicionário.

ARQUIVOS TXT

O que é um arquivo `.txt`?

Um arquivo `.txt` é texto puro.

Exemplo:

```
Ana  
Bruno  
Carla  
Diego  
Eduarda
```

Cada linha é uma sequência de caracteres.

Para o Python, tudo começa como `texto (str)`.

Abrindo arquivo do jeito certo

A forma recomendada é usar `with open(...)`.

```
with open("dados/nomes.txt", "r", encoding="utf-8") as arquivo:
    conteudo = arquivo.read()

print(conteudo)
```

Por quê?

- `with` fecha o arquivo automaticamente
- `"r"` significa leitura (*read*)
- `encoding="utf-8"` evita problemas com acentos

 Caminhos relativos e absolutos

Onde o Python procura o arquivo? **Na pasta de onde você está executando o script.**

Tipo	Exemplo	Quando usar
Relativo	"dados/nomes.txt"	Arquivo dentro da pasta do projeto. Use este na maioria dos casos.
Relativo "subir"	".../dados/nomes.txt"	Subir uma pasta antes de descer.
Absoluto Linux/Mac	"/home/ana/projeto/dados/nomes.txt"	Caminho fixo, da raiz.
Absoluto Windows	r"C:\Users\Ana\projeto\dados.txt"	Idem no Windows (o r antes da string evita problemas com \).

 Erro **mais comum** da aula: rodar o script de uma pasta errada e o Python não encontrar o arquivo.

Dica VSCode: abra a **pasta do projeto** (não só o arquivo solto) com **File** → **Open Folder**. O terminal vai abrir já no diretório certo.

🔗 Diagnóstico: `os.getcwd()`

Caminho relativo procura o arquivo a partir da **pasta de trabalho** (em inglês *current working directory*, CWD). Para descobrir qual é, pergunte ao Python:

Código:

```
import os
print("Pasta atual:", os.getcwd())

# alternativa moderna com pathlib
from pathlib import Path
print("Pasta atual:", Path.cwd())
```

Saída (exemplo):

```
Pasta atual: /home/aluno/Aula_Manipulacao_Arquivos
Pasta atual: /home/aluno/Aula_Manipulacao_Arquivos
```

Se a pasta impressa **não** terminar em **Aula_Manipulacao_Arquivos**, o `open("dados/x.csv")` vai falhar. Feche o VSCode e reabra com *Open Folder* na pasta certa.

⚠ `os.chdir()` existe, mas não use no código

O Python tem também `os.chdir("/caminho")` para **mudar** a pasta de trabalho.

```
import os
os.chdir("/home/ana/Aula_Manipulacao_Arquivos") # evite
```

Por que evitar:

- O caminho é específico da **sua máquina**. Na máquina do colega, o caminho é outro e o programa quebra.
- Mascara o problema real (você está rodando o script da pasta errada) em vez de resolvê-lo.
- Em projetos maiores, dois trechos de código mudando a pasta brigam entre si.

A solução certa: abrir o VSCode na pasta correta com *Open Folder* e usar **caminhos relativos** ("`dados/x.csv`").
Aí o programa funciona em qualquer máquina.

Use `os.getcwd()` para **diagnosticar**. Não use `os.chdir()` para **consertar**.

Por que `encoding="utf-8"`?

Computadores guardam texto como **números**. Cada conjunto de regras "número ↔ caractere" é uma **codificação**.

Codificação	Cobre	Problema
ASCII	só letras A–Z, dígitos, sinais básicos	sem acento, sem ç
Latin-1 / cp1252	acentos do português	quebra com emoji, japonês, árabe
UTF-8	praticamente todos os idiomas + emoji	padrão da web hoje

Sem `encoding="utf-8"`, o Python usa o padrão do **sistema operacional** (Windows: cp1252; Linux/Mac: UTF-8). Resultado: o mesmo código funciona na sua máquina e quebra na do colega.

```
# Sempre que abrir arquivo de texto, escreva:  
with open("arquivo.txt", "r", encoding="utf-8") as arquivo:  
    ...
```

Hábito útil: sempre escrever `encoding="utf-8"`, mesmo em arquivos sem acento. Custa nada e evita bugs estranhos.

► Lendo um arquivo inteiro

Código:

```
with open("dados/nomes.txt", "r", encoding="utf-8") as arquivo:  
    conteudo = arquivo.read()  
  
print(conteudo)
```

Saída:

```
## Ana  
## Bruno  
## Carla  
## Diego  
## Eduarda
```

≡ Lendo linha por linha

Em muitos problemas, queremos processar uma linha por vez.

```
with open("dados/nomes.txt", "r", encoding="utf-8") as arquivo:  
    for linha in arquivo:  
        nome = linha.strip()  
        print("Aluno:", nome)
```

```
## Aluno: Ana  
## Aluno: Bruno  
## Aluno: Carla  
## Aluno: Diego  
## Aluno: Eduarda
```

`strip()` remove `\n` e espaços sobrando no começo/fim da linha.

O problema do `\n`

Cada linha lida vem com um caractere **invisível** `\n` no final. Para enxergá-lo, usamos `repr()` (mostra a string "crua") e `len()` (conta os caracteres):

Código:

```
linha = "Ana\n"      # exatamente como vem do arquivo

print("Sem strip ->", repr(linha),          "| tamanho:", len(linha))
print("Com strip ->", repr(linha.strip()),  "| tamanho:", len(linha.strip()))
```

Saída:

```
## Sem strip -> 'Ana\n' | tamanho: 4
```

```
## Com strip -> 'Ana' | tamanho: 3
```

'Ana\n' tem **4** caracteres (A, n, a e a quebra). Depois de `strip()`, sobram **3**. Regra prática: ao ler linhas de `.txt`, quase sempre use `strip()`.

 Escrevendo em arquivo

Modo "w" cria um arquivo novo ou sobrescreve o arquivo existente.

```
with open("saida/nomes_saida.txt", "w", encoding="utf-8") as arquivo:  
    arquivo.write("Ana\n")  
    arquivo.write("Bruno\n")  
    arquivo.write("Carla\n")
```

```
## 4  
## 6  
## 6
```

```
## Ana  
## Bruno  
## Carla
```

+ Acrescentando ao final

Modo "a" acrescenta ao final do arquivo.

```
with open("saida/nomes_saida.txt", "a", encoding="utf-8") as arquivo:  
    arquivo.write("Diego\n")  
    arquivo.write("Eduarda\n")
```

```
## 6  
## 8
```

```
## Ana  
## Bruno  
## Carla  
## Diego  
## Eduarda
```

☐ Modos mais usados

Modo	Nome	O que faz
"r"	read	lê um arquivo existente
"w"	write	cria/sobrescreve um arquivo
"a"	append	acrescenta ao final

Cuidado: "w" apaga o conteúdo anterior se o arquivo já existir.

⚠ Erros comuns ao abrir arquivos

Erro	O que aconteceu	Como resolver
<code>FileNotFoundError: [Errno 2]</code>	O caminho está errado ou você está rodando o script de outra pasta.	Confira o <code>pwd/cd</code> . Use <i>Open Folder</i> no VSCode.
<code>PermissionError</code>	Sem permissão de leitura/escrita, ou arquivo aberto no Excel.	Feche o arquivo em outros programas; verifique permissões.
<code>UnicodeDecodeError</code>	O arquivo não está em UTF-8 (provavelmente Latin-1 / cp1252).	Tente <code>encoding="latin-1"</code> ou salve o arquivo de novo como UTF-8.
<code>IsADirectoryError</code>	Você passou o nome de uma pasta, não de um arquivo.	Acrescente o nome do arquivo no final do caminho.
Arquivo aparece vazio depois de escrever	Esqueceu o <code>with</code> (não fechou) ou o <code>\n</code> .	Sempre use <code>with open(...) as arquivo:</code> .

Quando um desses erros aparecer, leia a última linha do traceback. Ela diz exatamente o tipo do erro.

 Pergunta rápida

Considere que `dados/nomes.txt` contém:

Ana
Bruno
Carla

E o seguinte programa:

```
nomes = []  
with open("dados/nomes.txt", "r", encoding="utf-8") as arquivo:  
    for linha in arquivo:  
        nomes.append(linha)  
  
print(len(nomes))  
print(nomes[0] == "Ana")
```

Qual é a saída? Por quê?

 **Dica:** lembre do que vimos sobre o `\n`.

✓ Resposta

Saída:

3

False

- `len(nomes)` é **3**: três linhas → três strings na lista.
- `nomes[0] == "Ana"` é **False**: a primeira string é na verdade `"Ana\n"` (com a quebra de linha). Sem `strip()`, a comparação falha.

Correção:

```
nomes.append(linha.strip())
```

Esse bug roda sem erro mas dá resultado errado. Por isso vale a regra: use `strip()` por padrão ao ler `.txt`.

SETUP INICIAL

⬇️ Antes das práticas: baixar o pacote da aula

Para as práticas a seguir vocês vão precisar dos arquivos de apoio (**alunos.csv**, **produtos.csv**, **vendas.csv**, **nomes.txt**, entre outros).

Passos:

1. Abram o **Google Drive da disciplina** e baixem o arquivo **Aula_Manipulacao_Arquivos_Pacote.zip**.
2. Descompactem o **.zip** em uma pasta de fácil acesso (sugestão: **Documentos/aulas/manipulacao_arquivos**).
3. Confiram que aparecem as subpastas **dados/** e **saida/**.

Arquivo frutas.txt (Prática 1): vocês mesmos vão criar como parte do exercício, para treinar escrita de arquivo.

Os arquivos CSV já vêm prontos no pacote porque digitar dezenas de linhas à mão atrasaria a aula e introduziria erros de separadores e aspas.

Estrutura de pastas esperada

Depois de descompactar, a pasta da aula deve ficar assim:

Aula_Manipulacao_Arquivos/

```
├── dados/
│   ├── alunos.csv
│   ├── alunos_turmas.csv
│   ├── emprestimos.csv
│   ├── nomes.txt
│   ├── produtos.csv
│   └── vendas.csv
├── saida/
│   (vazia, vocês vão gerar arquivos aqui)
└── pratica1.py      <- vocês criam
```

Regras simples:

- Tudo o que **entra** no programa fica em **dados/**.
- Tudo o que o programa **gera** fica em **saida/**.
- Os scripts **.py** ficam **na raiz** da pasta (no mesmo nível de **dados/** e **saida/**).

Essa organização não é cosmética: ela é o que faz `open("dados/alunos.csv")` funcionar em qualquer máquina.

Abrindo o projeto no VSCode

Passo a passo (faça uma vez no início da aula):

1. Abra o **VSCode**.
2. Menu **File** → **Open Folder**.
3. Selecione a pasta **Aula_Manipulacao_Arquivos** (a que contém **dados/**). Não selecione um arquivo solto.
4. Abra um terminal integrado pelo menu **Terminal** → **New Terminal**.
5. No terminal, confira que está na pasta certa:

```
pwd          # Linux / Mac
cd           # Windows (sem argumento)
ls           # Linux / Mac (deve listar 'dados', 'saida')
dir          # Windows
```

1. Crie o arquivo **pratica1.py** com **File** → **New File** e salve dentro da pasta **Aula_Manipulacao_Arquivos**.

A partir daqui, qualquer **open("dados/x.csv")** no seu código vai funcionar.

 Prática rápida: **.txt**

Em dupla.

1. Crie um arquivo **frutas.txt** com 5 frutas, uma por linha.
2. Escreva um programa que leia o arquivo e mostre:
 - todas as frutas;
 - a quantidade de frutas;
 - a primeira fruta da lista.
3. Salve um arquivo **resumo_frutas.txt** com:

Quantidade de frutas: 5
Primeira fruta: banana

Dica:

```
frutas = []  
  
with open("frutas.txt", "r", encoding="utf-8") as arquivo:  
    for linha in arquivo:  
        frutas.append(linha.strip())
```


ARQUIVOS CSV

O que é CSV?

CSV significa **Comma-Separated Values**: valores separados por vírgula.

```
nome,matricula,nota1,nota2  
Ana,2026001,8.0,7.5  
Bruno,2026002,5.0,6.0  
Carla,2026003,9.0,8.5
```

É uma forma simples de representar uma tabela.

- Primeira linha: cabeçalho
- Linhas seguintes: registros
- Cada coluna vira um campo

Por que CSV é tão comum?

- **Universal:** abre em qualquer planilha (Excel, Google Sheets, LibreOffice) e em qualquer linguagem (Python, R, JavaScript, Java, Excel via macro).
- **Texto puro:** dá para abrir no Bloco de Notas e ler sem ferramenta nenhuma.
- **Leve:** um CSV de 10 mil linhas raramente passa de 1 MB.
- **Versionável:** o Git acompanha mudanças linha a linha, como em código.
- **Onde aparece:**
 - Datasets do Kaggle (machine learning) → quase sempre **.csv**
 - Dados abertos do governo (IBGE, dados.gov.br, TSE)
 - Exportações de sistemas (e-commerce, ERPs, CRMs)
 - Saída de relatórios financeiros e contábeis
 - Logs estruturados de aplicações

Quem domina CSV consegue trabalhar com a maioria dos datasets reais.

⚠ Não trate CSV na mão

Parece tentador fazer:

```
partes = linha.split(",")
```

Mas CSV pode ter detalhes chatos:

- vírgulas dentro de textos
- aspas
- linhas vazias
- quebras de linha
- diferentes formatos exportados por planilhas

Use o módulo **csv**. Ele existe para resolver esses detalhes.

✂ Primeiro contato com `csv.reader`

`csv.reader` lê cada linha como uma lista.

```
import csv

with open("dados/alunos.csv", "r", encoding="utf-8") as arquivo:
    leitor = csv.reader(arquivo)

    for linha in leitor:
        print(linha)
```

```
## ['nome', 'matricula', 'nota1', 'nota2']
## ['Ana', '2026001', '8.0', '7.5']
## ['Bruno', '2026002', '5.0', '6.0']
## ['Carla', '2026003', '9.0', '8.5']
## ['Diego', '2026004', '3.5', '4.0']
## ['Eduarda', '2026005', '7.0', '6.5']
## ['Felipe', '2026006', '10.0', '9.0']
```

 Melhor para nossa aula: **csv.DictReader**

DictReader usa o cabeçalho do CSV como chaves.

```
import csv

with open("dados/alunos.csv", "r", encoding="utf-8") as arquivo:
    leitor = csv.DictReader(arquivo)

    for aluno in leitor:
        print(aluno)
```

```
## {'nome': 'Ana', 'matricula': '2026001', 'nota1': '8.0', 'nota2': '7.5'}
## {'nome': 'Bruno', 'matricula': '2026002', 'nota1': '5.0', 'nota2': '6.0'}
## {'nome': 'Carla', 'matricula': '2026003', 'nota1': '9.0', 'nota2': '8.5'}
## {'nome': 'Diego', 'matricula': '2026004', 'nota1': '3.5', 'nota2': '4.0'}
## {'nome': 'Eduarda', 'matricula': '2026005', 'nota1': '7.0', 'nota2': '6.5'}
## {'nome': 'Felipe', 'matricula': '2026006', 'nota1': '10.0', 'nota2': '9.0'}
```

Agora cada linha já parece o dicionário que vimos na aula anterior.

Guardando em lista de dicionários

Na prática, queremos guardar os registros para processar depois.

```
import csv

alunos = []

with open("dados/alunos.csv", "r", encoding="utf-8") as arquivo:
    leitor = csv.DictReader(arquivo)

    for linha in leitor:
        alunos.append(linha)

for aluno in alunos[:2]:          # mostra os 2 primeiros
    print(aluno)
print("Total:", len(alunos))

## {'nome': 'Ana', 'matricula': '2026001', 'nota1': '8.0', 'nota2': '7.5'}
## {'nome': 'Bruno', 'matricula': '2026002', 'nota1': '5.0', 'nota2': '6.0'}

## Total: 6
```

⇔ Tudo vem como texto

Quando lemos de arquivo, os valores vêm como `str`. Veja o que acontece:

```
aluno = {"nome": "Ana", "nota1": "8.0", "nota2": "7.5"}  
  
nota1 = aluno["nota1"]  
nota2 = aluno["nota2"]  
  
print("nota1 + nota2 =", nota1 + nota2)    # concatena strings!  
print((nota1 + nota2) / 2)                 # TypeError
```

Saída:

```
## nota1 + nota2 = 8.07.5
```

```
## TypeError: unsupported operand type(s) for /: 'str' and 'int'
```

Precisamos converter:

```
nota1 = float(aluno["nota1"])  
nota2 = float(aluno["nota2"])  
media = (nota1 + nota2) / 2
```


 Calculando média dos alunos

```
import csv

alunos = []

with open("dados/alunos.csv", "r", encoding="utf-8") as arquivo:
    leitor = csv.DictReader(arquivo)

    for aluno in leitor:
        nota1 = float(aluno["nota1"])
        nota2 = float(aluno["nota2"])
        media = (nota1 + nota2) / 2

        aluno["media"] = media
        alunos.append(aluno)

for aluno in alunos:
    print(aluno["nome"], aluno["media"])
```

```
## Ana 7.75
## Bruno 5.5
## Carla 8.75
## Diego 3.75
## Eduarda 6.75
## Felipe 9.5
```

👉 Classificando a situação

Regra:

- média ≥ 7 : **Aprovado**
- média ≥ 4 e < 7 : **Recuperacao**
- média < 4 : **Reprovado**

```
def classificar(media):  
    if media  $\geq$  7:  
        return "Aprovado"  
    elif media  $\geq$  4:  
        return "Recuperacao"  
    else:  
        return "Reprovado"
```

Separar a classificação em função deixa o programa mais legível.

⚙️ Processando alunos com função

```
def classificar(media):  
    if media >= 7:  
        return "Aprovado"  
    elif media >= 4:  
        return "Recuperacao"  
    else:  
        return "Reprovado"  
  
for aluno in alunos:  
    nota1 = float(aluno["nota1"])  
    nota2 = float(aluno["nota2"])  
    media = (nota1 + nota2) / 2  
  
    aluno["media"] = media  
    aluno["situacao"] = classificar(media)
```

Agora cada dicionário tem campos novos calculados pelo programa.

> Resultado parcial

```
for aluno in alunos:
    print(
        aluno["nome"],
        "- média:",
        round(aluno["media"], 2),
        "-",
        aluno["situacao"]
    )
```

```
## Ana - média: 7.75 - Aprovado
## Bruno - média: 5.5 - Recuperacao
## Carla - média: 8.75 - Aprovado
## Diego - média: 3.75 - Reprovado
## Eduarda - média: 6.75 - Recuperacao
## Felipe - média: 9.5 - Aprovado
```

PRÁTICA 2

 Prática: produtos em CSV

Em dupla.

Use `dados/produtos.csv`.

1. Leia o arquivo com `csv.DictReader`.
2. Guarde os produtos em uma lista de dicionários.
3. Converta `preco` para `float`.
4. Converta `quantidade` para `int`.
5. Mostre:
 - nome de cada produto;
 - valor total daquele item em estoque (`preco * quantidade`);
 - valor total geral do estoque.

Dica:

```
produto["preco"] = float(produto["preco"])
produto["quantidade"] = int(produto["quantidade"])
```

Pergunta rápida

Considere o trecho:

```
import csv

with open("dados/produtos.csv", "r", encoding="utf-8") as arquivo:
    leitor = csv.DictReader(arquivo)
    primeiro = next(leitor)

print(type(primeiro))
print(type(primeiro["preco"]))
print(primeiro["preco"] + 1)
```

Sabendo que `produtos.csv` tem cabeçalho `nome,preco,quantidade` e a primeira linha é `Caderno,18.50,10`, o que sai na tela?

▮ Pense em cada `print` antes de rodar.

✓ Resposta

```
<class 'dict'>  
<class 'str'>  
TypeError: can only concatenate str (not "int") to str
```

- `primeiro` é um **dicionário** (o `DictReader` transforma cada linha em `dict`).
- `primeiro["preco"]` é uma **string** (`"18.50"`). O `csv` lê tudo como texto.
- `"18.50" + 1` quebra: não dá para somar string com inteiro.

Para somar, converte primeiro:

```
print(float(primeiro["preco"]) + 1)    # 19.5
```

Esquecer da conversão é o erro mais comum ao começar com CSV.

GERANDO ARQUIVOS DE SAÍDA

 Saída em TXT ou CSV?

Use `.txt` quando quiser um resumo para leitura humana:

Total de alunos: 6

Aprovados: 3

Média geral: 6.83

Use `.csv` quando quiser uma tabela:

```
nome,matricula,media,situacao  
Ana,2026001,7.75,Aprovado  
Bruno,2026002,5.5,Recuperacao
```

Escrevendo CSV com DictWriter

```
import csv

campos = ["nome", "matricula", "media", "situacao"]

with open("saida/resultado_alunos.csv", "w", encoding="utf-8", newline="") as arquivo:
    escritor = csv.DictWriter(arquivo, fieldnames=campos)

    escritor.writeheader()

    for aluno in alunos:
        escritor.writerow({
            "nome": aluno["nome"],
            "matricula": aluno["matricula"],
            "media": round(aluno["media"], 2),
            "situacao": aluno["situacao"]
        })
```

`newline=""` evita linhas em branco extras em alguns sistemas.

► Gerando `resultado_alunos.csv`

```
campos = ["nome", "matricula", "media", "situacao"]

with open("saida/resultado_alunos.csv", "w", encoding="utf-8", newline="") as arquivo:
    escritor = csv.DictWriter(arquivo, fieldnames=campos)
    escritor.writeheader()

    for aluno in alunos:
        escritor.writerow({
            "nome": aluno["nome"],
            "matricula": aluno["matricula"],
            "media": round(aluno["media"], 2),
            "situacao": aluno["situacao"]
        })
```

```
## 31
## 27
## 31
## 29
## 30
## 34
## 29
```

```
## nome,matricula,media,situacao
## Ana,2026001,7.75,Aprovado
## Bruno,2026002,5.5,Recuperacao
## Carla,2026003,8.75,Aprovado
## Diego,2026004,3.75,Reprovado
## Eduarda,2026005,6.75,Recuperacao
## Felipe,2026006,9.5,Aprovado
```

Gerando resumo

Antes de escrever o resumo, calculamos os indicadores.

```
total = len(alunos)
aprovados = 0
recuperacao = 0
reprovados = 0
soma_medias = 0
melhor_aluno = alunos[0]

for aluno in alunos:
    soma_medias += aluno["media"]

    if aluno["situacao"] == "Aprovado":
        aprovados += 1
    elif aluno["situacao"] == "Recuperacao":
        recuperacao += 1
    else:
        reprovados += 1

    if aluno["media"] > melhor_aluno["media"]:
        melhor_aluno = aluno

media_geral = soma_medias / total
```

 Salvando o resumo em **.txt**

```
with open("saida/resumo_alunos.txt", "w", encoding="utf-8") as arquivo:
    arquivo.write(f"Total de alunos: {total}\n")
    arquivo.write(f"Aprovados: {aprovados}\n")
    arquivo.write(f"Recuperacao: {recuperacao}\n")
    arquivo.write(f"Reprovados: {reprovados}\n")
    arquivo.write(f"Media geral: {media_geral:.2f}\n")
    arquivo.write(f"Maior media: {melhor_aluno['nome']} - {melhor_aluno['media']:.2f}\n")
```

```
## 19
## 13
## 15
## 14
## 18
## 27
```

```
## Total de alunos: 6
## Aprovados: 3
## Recuperacao: 2
## Reprovados: 1
## Media geral: 7.00
## Maior media: Felipe - 9.50
```

ORGANIZANDO EM FUNÇÕES

Por que quebrar em funções?

Sem funções, o programa vira um bloco grande:

```
ler arquivo  
converter campos  
calcular média  
classificar  
contar situações  
salvar CSV  
salvar TXT
```

Com funções, cada parte ganha um nome:

```
alunos = ler_alunos("dados/alunos.csv")  
processar_alunos(alunos)  
salvar_resultado(alunos, "saida/resultado.csv")  
salvar_resumo(alunos, "saida/resumo.txt")
```

Isso facilita leitura, teste e correção.

</> Estrutura recomendada

```
import csv

def ler_alunos(nome_arquivo):
    alunos = []
    with open(nome_arquivo, "r", encoding="utf-8") as arquivo:
        leitor = csv.DictReader(arquivo)
        for linha in leitor:
            alunos.append(linha)
    return alunos

def classificar(media):
    if media >= 7:
        return "Aprovado"
    elif media >= 4:
        return "Recuperacao"
    else:
        return "Reprovado"
```

</> Função de processamento

```
def processar_alunos(alunos):  
    for aluno in alunos:  
        nota1 = float(aluno["nota1"])  
        nota2 = float(aluno["nota2"])  
        media = (nota1 + nota2) / 2  
  
        aluno["media"] = media  
        aluno["situacao"] = classificar(media)
```

Essa função altera cada dicionário da lista, adicionando:

- `media`
- `situacao`

Função `salvar_resultado` (CSV)

```
def salvar_resultado(alunos, nome_arquivo):  
    campos = ["nome", "matricula", "media", "situacao"]  
  
    with open(nome_arquivo, "w", encoding="utf-8", newline="") as arquivo:  
        escritor = csv.DictWriter(arquivo, fieldnames=campos)  
        escritor.writeheader()  
  
        for aluno in alunos:  
            escritor.writerow({  
                "nome": aluno["nome"],  
                "matricula": aluno["matricula"],  
                "media": round(aluno["media"], 2),  
                "situacao": aluno["situacao"]  
            })
```

Recebe a lista de alunos já processada e o nome do arquivo de saída.

 Função **salvar_resumo** (TXT)

```
def salvar_resumo(alunos, nome_arquivo):
    total = len(alunos)
    aprovados = 0
    recuperacao = 0
    reprovados = 0
    soma_medias = 0
    melhor_aluno = alunos[0]

    for aluno in alunos:
        soma_medias += aluno["media"]
        if aluno["situacao"] == "Aprovado":
            aprovados += 1
        elif aluno["situacao"] == "Recuperacao":
            recuperacao += 1
        else:
            reprovados += 1
        if aluno["media"] > melhor_aluno["media"]:
            melhor_aluno = aluno

    media_geral = soma_medias / total

    with open(nome_arquivo, "w", encoding="utf-8") as arquivo:
        arquivo.write(f"Total de alunos: {total}\n")
        arquivo.write(f"Aprovados: {aprovados}\n")
        arquivo.write(f"Recuperacao: {recuperacao}\n")
        arquivo.write(f"Reprovados: {reprovados}\n")
        arquivo.write(f"Media geral: {media_geral:.2f}\n")
        arquivo.write(f"Maior media: {melhor_aluno['nome']} - {melhor_aluno['media']:.2f}\n")
```

</> Programa principal

```
def main():  
    alunos = ler_alunos("dados/alunos.csv")  
    processar_alunos(alunos)  
    salvar_resultado(alunos, "saida/resultados_alunos.csv")  
    salvar_resumo(alunos, "saida/resumo_alunos.txt")  
    print("Arquivos gerados com sucesso.")  
  
if __name__ == "__main__":  
    main()
```

Esse padrão separa bem entrada, processamento e saída.

PRÁTICA 3

☰ Miniatividade orientada: vendas

Em dupla.

Use `dados/vendas.csv`.

Crie um programa que:

1. Leia as vendas com `csv.DictReader`.
2. Converta `quantidade` para `int`.
3. Converta `valor_unitario` para `float`.
4. Calcule o total de cada venda.
5. Gere `saida/relatorio_vendas.csv` com:

`produto, categoria, quantidade, valor_unitario, total`

1. Gere `saida/resumo_vendas.txt` com:
 - total geral vendido
 - maior venda
 - total vendido por categoria

</> Miniatividade: funções sugeridas

Funções sugeridas:

```
def ler_vendas(nome_arquivo):  
    pass  
  
def processar_vendas(vendas):  
    pass  
  
def salvar_relatorio(vendas, nome_arquivo):  
    pass  
  
def salvar_resumo(vendas, nome_arquivo):  
    pass
```


COMPARANDO FORMATOS

 .txt vs .csv vs .json

Característica	.txt	.csv	.json
Estrutura	livre, linha de texto	tabela (linhas × colunas)	árvore (dict/list aninhados)
Tipos de dado	só str	só str (você converte)	preserva int, float, bool, null
Hierarquia	não	não	sim (aninhamento natural)
Excel abre?	sim	sim, como planilha	não (precisa de outro app)
Tamanho	mínimo	pequeno	um pouco maior (tem chaves)
Ferramentas Python	open	csv.DictReader/Writer	json.load/dump
Uso típico	logs, listas simples	tabelas, datasets, exports	APIs, configs, dados aninhados

Regra rápida: tabela → CSV. Dados com hierarquia → JSON. Texto livre → TXT.

👁️ Espiando JSON (apenas para conhecer)

JSON guarda o mesmo aluno que estamos usando, mas mantém os tipos e aceita aninhamento:

```
{  
  "nome": "Ana",  
  "matricula": 2026001,  
  "notas": [8.0, 7.5],  
  "endereco": {"cidade": "Londrina", "uf": "PR"}  
}
```

Em Python:

```
import json  
with open("aluno.json", "r", encoding="utf-8") as arquivo:  
    aluno = json.load(arquivo)  
print(aluno["notas"][0])      # 8.0 (já vem float!)
```

Não vamos cobrar JSON na A2. Fica como leitura para o Trabalho Final, já que várias APIs e arquivos de configuração usam esse formato.

BOAS PRÁTICAS

☑ Boas práticas com arquivos

- **Sempre** abra com `with open(...)`. Garante que o arquivo é fechado mesmo se der erro.
- **Sempre** passe `encoding="utf-8"`. Evita surpresas entre Windows, Mac e Linux.
- Use **caminhos relativos** quando os arquivos estão na pasta do projeto ("**dados/x.csv**"), não absolutos.
- Separe **entrada, processamento e saída** em funções diferentes. Cada parte fica fácil de testar.
- Para CSV, prefira **DictReader/DictWriter**. É mais legível que acessar por índice numérico.
- **Converta** tipos logo depois de ler (**float, int**). Não deixe a conversão espalhada pelo código.
- No **DictWriter**, use `newline=""` ao abrir o arquivo. Evita linhas em branco no Windows.
- Mantenha uma pasta **dados/** para entradas e **saida/** para saídas. Nunca sobrescreva a entrada.
- Antes de rodar com `"w"`, confirme o nome. `"w"` **apaga** o que existia.
- **Nunca** edite o CSV de entrada no Excel sem cuidado (ele pode mudar separadores, vírgulas decimais, encoding).

Esses hábitos deixam o programa fácil de rodar em outras máquinas.

Resumo da aula

- Arquivos permitem guardar dados depois que o programa fecha.
- Use `with open(...)` para abrir arquivos.
- Use `"r"` para ler, `"w"` para escrever e `"a"` para acrescentar.
- Use `strip()` para remover quebras de linha em arquivos `.txt`.
- Use o módulo `csv` para trabalhar com arquivos `.csv`.
- `csv.DictReader` transforma cada linha em dicionário.
- Dados lidos de arquivo vêm como texto: converta com `int()` e `float()`.
- `csv.DictWriter` gera arquivos CSV com cabeçalho.
- Relatórios simples podem ser salvos em `.txt`.
- Organizar em funções separa entrada, processamento e saída.
- `.csv` é para tabela; `.json` para dados aninhados; `.txt` para texto livre.
- `encoding="utf-8"` é hábito **obrigatório**. Evita bugs por diferença entre sistemas operacionais.

🔍 Checklist de depuração

Quando der erro, percorra esta lista **em ordem**:

Caminho e abertura

- O arquivo está na pasta correta? Confira `pwd` no terminal.
- O nome do arquivo está escrito **exatamente** igual (maiúsculas, acentos)?
- Usou `encoding="utf-8"`? Sem isso, dá `UnicodeDecodeError` em alguns sistemas.

Leitura

- O CSV tem cabeçalho na primeira linha?
- As chaves usadas no código existem no cabeçalho? (`aluno["nota1"]` precisa de coluna `nota1`).
- Esqueceu de usar `strip()` em linhas de `.txt`?

Conversão

- O campo numérico foi convertido para `int` ou `float`?
- Tem campo vazio ou com texto onde devia ter número?

Escrita

- O arquivo de saída está sendo criado na pasta certa? (a pasta **precisa existir**).
- No `DictWriter`, passou `newline=""` ao abrir o arquivo?
- Está abrindo com `"w"` sem querer (sobrescreve) quando queria `"a"`?

Estrutura

- O programa fecha o arquivo? Usou `with`?
- O erro aponta para leitura, conversão, cálculo ou escrita? Isso isola onde está o bug.

Referências

- FORBELLONE, A. L. V.; EBERSPACHER, H. F. **Lógica de Programação**: A Construção de Algoritmos e Estruturas de Dados. 4. ed. São Paulo: Pearson, 2016.
- MENEZES, N. N. C. **Introdução a Programação com Python**: Algoritmos e Lógica de Programação para Iniciantes. 4. ed. Rio de Janeiro: Novatec, 2024.
- MCKINNEY, W. **Python para Análise de Dados**: Tratamento de Dados com Pandas, NumPy e Jupyter. 3. ed. São Paulo: Novatec, 2023. (*capítulo sobre I/O de arquivos*)
- Documentação Python, leitura e escrita de arquivos: docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files
- Documentação Python, módulo **csv**: docs.python.org/3/library/csv.html
- Documentação Python, módulo **json**: docs.python.org/3/library/json.html
- PEP 8, Style Guide for Python Code: peps.python.org/pep-0008
- Real Python, **Working With Files in Python**: realpython.com/working-with-files-in-python
- Dados abertos do governo federal: dados.gov.br

LISTA DE EXERCÍCIOS (A2)

☰ Para casa: Lista A2

Os exercícios de **Dicionários + Arquivos** que preparam vocês para a **Avaliação A2 (09/06)** estão no arquivo:

| Lista_Exercicios_Dicionarios_Arquivos_A2.pdf

disponível na pasta da aula no Drive.

- A lista combina os dois temas: ler de arquivo, montar lista de dicionários, processar e gerar saída.
- Os arquivos de dados (**alunos.csv**, **produtos.csv**, **vendas.csv**, **alunos_turmas.csv**, **emprestimos.csv**, **frutas.txt**, **nomes.txt**) estão na subpasta **dados/**.
- Tragam **dúvidas** na próxima aula. Vamos ter um momento de revisão antes da prova.

TRABALHO FINAL (A1)

Apresentação do Trabalho Final (A1)

A partir desta aula, vocês começam o **Projeto Final** que vale a **nota A1**.

Defesa: 16/06/2026 (Aula 15).

Em **duplas**, vocês vão construir um programa em Python que:

1. **Lê** dados de um arquivo (**.csv** ou **.txt**);
2. **Processa** os dados usando os recursos vistos no semestre (listas, dicionários, funções);
3. **Gera** ao menos um arquivo de saída (relatório **.txt** ou tabela **.csv**);
4. É **organizado em funções** (não pode ser um único bloco gigante).

Detalhes completos (tema livre, critérios, rubrica) estão no documento **Projeto_Final_Programacao_de_Computadores.pdf** no Drive. Confirmem a dupla até a próxima aula.

PRÓXIMA AULA

📌 Próxima aula: Avaliação A2 (09/06)

A próxima aula (09/06) é a Prova A2.

Conteúdo cumulativo, com foco em:

- Condicionais e repetição
- Funções (**def**, parâmetros, **return**)
- Listas, tuplas e matrizes
- **Dicionários** (foco maior)
- **Manipulação de arquivos .txt e .csv** (foco maior)

Como se preparar:

- Resolver a **Lista_Exercicios_Dicionarios_Arquivos_A2.pdf**
- Rever os exemplos rodados em sala
- Trazer dúvidas. Vou reservar o começo da aula para revisão
- Verificar antes que o VSCode + Python estão funcionando na sua máquina

Obrigado!

Dúvidas?